SWINBURNE VIETNAM

HO CHI MINH CAMPUS

COS30018 - COS30018-Intelligent Systems

Week 9: Report

INSTRUCTOR: MR. KY TRUNG PHAM

STUDENT NAME: LÊ TIẾN DŨNG-SWS00683

HO CHI MINH CITY – MAR-2026

Contents

1. What is the "Ensemble" approach? How does combining a statistical model like ARIMA with a Deep Learning model like LSTM improve the reliability of the prediction? 3

2. Explain how you integrated ARIMA/SARIMA into your Python workflow. How do these traditional models handle "Seasonality" in stock data? 3

3. How do you resolve conflicts if the LSTM predicts a "Price Up" movement while the ARIMA model predicts a "Price Down" movement? 5

1. What is the "Ensemble" approach? How does combining a statistical model like ARIMA with a Deep Learning model like LSTM improve the reliability of the prediction?

An ensemble approach combines predictions from multiple models to improve robustness and reduce the risk of relying on a single model that may fail under certain market conditions. In this project, I combined a traditional statistical model (SARIMA) with a deep learning model (LSTM):

- LSTM is suitable for learning non-linear patterns and can use multiple features (Close, Volume, High, Low) to capture market behavior.
- SARIMA is a classical time-series model that provides a stable forecast based on statistical structure. It is often less sensitive to noise and can help as a secondary viewpoint.

By combining these two models, the system can become more reliable because each model contributes different strengths. The ensemble prediction is produced using a weighted average of the LSTM and SARIMA forecasts.

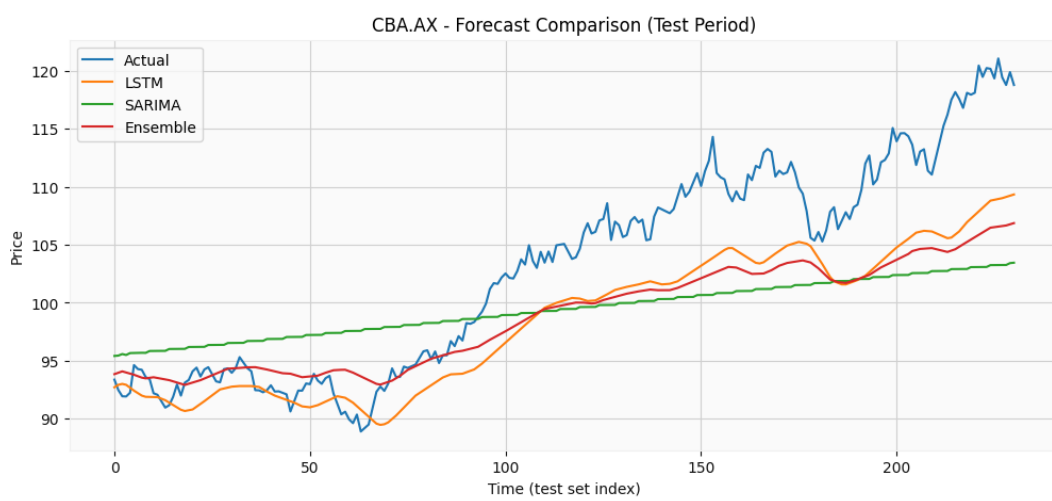

Figure 1: "Week 9 – CBA.AX Forecast Comparison (Actual vs LSTM vs SARIMA vs Ensemble)"

2. Explain how you integrated ARIMA/SARIMA into your Python workflow. How do these traditional models handle "Seasonality" in stock data?

To integrate SARIMA into my Python workflow, I used the statsmodels library (SARIMAX implementation). The main steps were:

1. Extract the training Close price series from the dataset.
2. Fit a SARIMA model on the training series using:
   - order = (1,1,1)

- seasonal_order = (1,1,1,5)

3. Forecast the entire test period (same length as the test set).
4. Evaluate SARIMA predictions using the same regression metrics as LSTM (MAE, MSE, RMSE).
5. Combine SARIMA predictions with LSTM predictions to form the ensemble forecast.

Seasonality handling:

SARIMA extends ARIMA by including seasonal parameters (P, D, Q, m). The m value represents the seasonal cycle length. In this experiment, I used m = 5 to represent a weekly trading cycle (5 trading days). This allows SARIMA to model repeating weekly patterns when they exist in the data.

Runtime results (efficiency of traditional model integration):

- SARIMA fit time: 0.554 seconds
- SARIMA forecast time: 0.005 seconds

This shows SARIMA can be integrated efficiently into the workflow with minimal computation cost compared to deep learning training.

Evaluation Results: LSTM vs SARIMA vs Ensemble (CBA.AX):

I evaluated all predictions on the same test period using MAE, MSE, and RMSE (lower values indicate better performance).

Model	**Train time (sec)**	**MAE**	**MSE**	**RMSE**
LSTM	87.66	**4.7703**	**32.7204**	**5.7202**
SARIMA	0.5538	6.5959	59.4250	7.7088
Ensemble (wLSTM=0.58, wSARIMA=0.42)	~0.56*	4.9872	40.0407	6.3278

```
                                Model  Train time (sec)       MAE  \
0                                LSTM         87.659988  4.770269
1                              SARIMA          0.553824  6.595935
2  Ensemble (w_lstm=0.58, w_sarima=0.42)         0.558916  4.987162

         MSE      RMSE
0  32.720400  5.720175
1  59.425048  7.708764
2  40.040729  6.327774
```

Figure 2: "Week 9 – Test Performance Table (MAE/MSE/RMSE): LSTM vs SARIMA vs Ensemble"

*The ensemble computation itself is almost instantaneous because it is only a weighted combination. Total system runtime includes LSTM training time plus SARIMA fit/forecast time.

Discussion:

- SARIMA was the fastest model to train but produced the largest prediction errors in this test period.
- LSTM achieved the best accuracy (lowest MAE/MSE/RMSE).
- The ensemble produced stable predictions but did not outperform LSTM in this experiment (ensemble MAE 4.99 > LSTM MAE 4.77). This outcome is still valid: ensemble methods improve robustness, but accuracy improvement depends on model selection and weight tuning.

3. How do you resolve conflicts if the LSTM predicts a "Price Up" movement while the ARIMA model predicts a "Price Down" movement?

To handle situations where the LSTM and SARIMA predictions disagree in direction, I implemented a direction-based conflict resolution rule:

- Convert each model's next-day prediction into a direction:
    - UP if predicted next price > last close
    - DOWN otherwise
- If both models agree, follow that direction.
- If they conflict, trust the model with better recent performance (lower MAE).

Observed conflict example (next-day forecast):

- Last close: 118.7782
- LSTM next prediction: 109.3372 → DOWN
- SARIMA next prediction: 118.9694 → UP
- Ensemble next prediction: 113.3797 → DOWN

Because LSTM had lower test MAE (4.77) compared to SARIMA (6.60), the final decision was:

Final decision: DOWN

Reason: conflict → trust the model with lower MAE (LSTM).

This rule ensures the decision process is consistent, transparent, and based on measured model performance.

```
[48]: # 4) Conflict resolution example (Next-day direction Up/Down)
# If they disagree, choose the model with lower MAE (simple, defensible for assignment).

full_close = pd.concat([train_close, test_close])
last_close = float(full_close.iloc[-1])

# LSTM next-day prediction should already exist as `prediction` from your LSTM cell
if 'prediction' in globals():
    lstm_next = float(np.asarray(prediction).reshape(-1)[0])
else:
    raise NameError("Missing `prediction` for LSTM next-day. Run your LSTM next-day prediction cell.")

# For next-day SARIMA forecast: fit SARIMA on all known data (train+test), then forecast 1 step
sarima_model_full = SARIMAX(
    full_close,
    order=ORDER,
    seasonal_order=SEASONAL_ORDER,
    enforce_stationarity=False,
    enforce_invertibility=False
).fit(disp=False)

sarima_next = float(sarima_model_full.get_forecast(steps=1).predicted_mean.iloc[0])
ensemble_next = (w_lstm * lstm_next) + (w_sarima * sarima_next)

def direction(pred, last):
    return "UP" if pred > last else "DOWN"

dir_lstm = direction(lstm_next, last_close)
dir_sarima = direction(sarima_next, last_close)
dir_ens = direction(ensemble_next, last_close)

print("Last close:", last_close)
print("LSTM next:", lstm_next, "| Direction:", dir_lstm)
print("SARIMA next:", sarima_next, "| Direction:", dir_sarima)
print("Ensemble next:", ensemble_next, "| Direction:", dir_ens)

if dir_lstm == dir_sarima:
    final_decision = dir_lstm
    reason = "Both models agree."
else:
    final_decision = dir_lstm if lstm_mae <= sarima_mae else dir_sarima
    reason = f"Conflict -> trust lower MAE model ({'LSTM' if lstm_mae <= sarima_mae else 'SARIMA'})."

print("FINAL decision:", final_decision)
print("Reason:", reason)
```

Figure 9.3: "Week 9 – Direction Conflict Resolution (LSTM vs SARIMA, Performance-Based Rule)"